Módulo `config`

> Código: `src/config.h` · Relacionados: 01 — Visão geral, 03 — Inclinação, 04 — Detecção e calibração, 05 — Sensores analógicos, 06 — Motor e NVS

Papel no sistema

Reúne em um lugar só tudo que é número fixo do firmware: pinos, endereços I2C, tempos, limiares dos sensores, padrões de vibração, nome e senha do Wi-Fi e os valores de fábrica dos parâmetros ajustáveis. Só tem `#define`, sem lógica. A ideia é que um ajuste de bancada (ex.: ruído do ADC maior que o esperado) seja uma troca de número aqui, sem procurar o valor espalhado pelo código.

Os `PADRAO_*` são só o ponto de partida: o que o usuário ajusta pela serial ou pelo app fica salvo na NVS (settings) e vale mais que estes valores.

Dependências

- Usa: `<stdint.h>`.
- É usado por: praticamente todo o firmware (`main.cpp`, `sensors.cpp`, `mpu.cpp`, `motor.cpp`, `posture.cpp`, `console.cpp`, `protocolo.cpp`, `web.cpp`) e pelo simulador (`sim/sim_main.cpp`).
- Roda em: compilação (não há código executável).

Pinagem dupla

O firmware atende as duas montagens existentes ao mesmo tempo, porque nenhum pino conflita:

Função	Constante	GPIO	montagem.pdf / GUIA	BioSpine V3
I2C SDA (MPU-6050)	`PIN_I2C_SDA`	21	sim	sim
I2C SCL (MPU-6050)	`PIN_I2C_SCL`	22	sim	sim
Flex 1	`PIN_FLEX`	34	sim	flex esquerdo
Flex 2	`PIN_FLEX2`	35	reserva	flex direito
FSR (pressão)	`PIN_FSR`	32	—	sim
Motor (via NPN)	`PIN_MOTOR`	25	sim	—
Botão de calibração	`PIN_BUTTON`	27	sim	—
LED azul da placa	`PIN_LED`	2	—	sim

Por que esses pinos:

- **Todo sensor analógico fica no ADC1 (GPIO 32–39).** O ADC2 do ESP32 não funciona com o Wi-Fi ligado.
- GPIO 34 e 35 são só entrada (servem para ADC, não para saída).
- Saídas evitam os pinos de boot (0, 2, 12, 15). O GPIO2 é usado só para o LED, que já existe na placa.
- O motor puxa ~60 mA: o GPIO não aguenta, por isso vai por um transistor NPN (resistor de 1 kΩ na base).
- Botão para o GND com `INPUT_PULLUP`: solto = `HIGH`, apertado = `LOW`. Dispensa resistor externo.

Tipos e constantes

MPU-6050

Constante	Valor	Unidade	Usada em	Descrição
`MPU_ADDR_A`	`0x68`	—	`mpu.cpp`, `console.cpp`	endereço com AD0 solto ou em GND (padrão do GY-521)
`MPU_ADDR_B`	`0x69`	—	`mpu.cpp`, `console.cpp`	endereço com AD0 em 3V3
`MPU_I2C_HZ`	100000	Hz	`sensors.cpp`	100 kHz: mais tolerante a fio longo no colete do que 400 kHz
`MPU_FILTRO_GIRO`	0.98	—	`sensors.cpp`	peso do giroscópio no filtro complementar (o acelerômetro fica com 0,02)
`MPU_REINICIO_MS`	2000	ms	`sensors.cpp`	intervalo entre tentativas de reconectar o MPU
`GIRO_PARADO_DPS`	5.0	°/s	`mpu.cpp`	variação máxima do giroscópio numa janela para considerar o sensor parado
`GIRO_OFFSET_MAX`	25.0	°/s	`mpu.cpp`	offset de giroscópio plausível; acima disso é movimento, não offset

Sensores analógicos (ADC de 12 bits, 0–4095)

Constante	Valor	Unidade	Usada em	Descrição
`ADC_MIN_VALIDO`	100	ADC	`sensors.cpp`, `main.cpp`, `console.cpp`	abaixo disso: flex solto ou quebrado
`ADC_MAX_VALIDO`	4000	ADC	`sensors.cpp`, `main.cpp`, `console.cpp`	acima disso: falta o resistor de 10 kΩ do divisor
`ADC_RUIDO_MAX`	300	ADC	`sensors.cpp`	amplitude (máx − mín) máxima na janela da média; pino solto oscila mais
`ADC_MEDIA_N`	10	amostras	`sensors.cpp`	tamanho da média móvel (10 amostras a 100 Hz = janela de 100 ms)
`FLEX_LIGA_MS`	2000	ms	`sensors.cpp`	tempo de leitura válida contínua para considerar um sensor presente (flex 1, flex 2 e FSR)
`FLEX_DESLIGA_MS`	1000	ms	`sensors.cpp`	tempo de leitura inválida contínua para considerar o flex ausente
`FSR_LIMIAR`	200	ADC	`sensors.cpp`	FSR com leitura estável acima disso por `FLEX_LIGA_MS` = sensor existe
`ADC_FANTASMA_DIF`	300	ADC	`sensors.cpp`	FSR com leitura a menos disso do flex 1 é tratado como "fantasma" (pino solto copiando o canal vizinho)

Motor (LEDC = periférico de PWM do ESP32)

Constante	Valor	Unidade	Usada em	Descrição
`MOTOR_LEDC_CANAL`	0	—	`motor.cpp`	canal LEDC (core Arduino 2.x; no 3.x o canal é escolhido pelo pino)
`MOTOR_LEDC_FREQ`	20000	Hz	`motor.cpp`	20 kHz: acima da faixa audível (o motor não "chia")
`MOTOR_LEDC_BITS`	8	bits	`motor.cpp`	resolução do duty: 0–255
`MOTOR_ALERTA_ON_MS`	300	ms	`motor.cpp`	alerta: tempo ligado de cada pulso

Constante	Valor	Unidade	Usada em	Descrição
`MOTOR_ALERTA_OFF_MS`	700	ms	`motor.cpp`	alerta: pausa entre pulsos (menos incômodo que vibração contínua)
`MOTOR_TESTE_MS`	1000	ms	`main.cpp`	pulso do comando `motor`, do botão "Testar motor" e do `diag`
`MOTOR_CONFIRMA_MS`	120	ms	`main.cpp`	pulso curto de aviso: boot (1), início da calibração (1), fim da calibração (2)
`MOTOR_ERRO_MS`	600	ms	`main.cpp`	pulso longo: calibração recusada

Tempos (agendamento por `millis()`, sem `delay()`)

Constante	Valor	Frequência	Usada em	Descrição
`PERIODO_MPU_MS`	5 ms	200 Hz	`sensors.cpp`	leitura do MPU e passo do filtro complementar
`PERIODO_ADC_MS`	10 ms	100 Hz	`sensors.cpp`	leitura dos três canais analógicos
`PERIODO_POSTURA_MS`	20 ms	50 Hz	`main.cpp` (`passoControle`)	cálculo do desvio e detecção
`PERIODO_ENVIO_MS`	100 ms	10 Hz	`main.cpp` (`loop`)	envio para o app e para o plotter
`CALIBRACAO_ESPERA_MS`	1500 ms	—	`main.cpp`	depois do clique: tempo para a pessoa se ajeitar
`CALIBRACAO_MS`	1000 ms	—	`main.cpp`	duração da média da calibração
`CALIBRACAO_GIRO_MAX`	30 °/s	—	`main.cpp`	se o giroscópio passar disso durante a média, a calibração é recusada
`BOTAO_DEBOUNCE_MS`	50 ms	—	`main.cpp`	tempo que o botão precisa ficar estável para contar

Wi-Fi

Constante	Valor	Usada em	Descrição
`WIFI_SSID`	`"BioSpine"`	`web.cpp`, `console.cpp`	nome da rede criada pelo colete (Access Point)
`WIFI_SENHA`	`"colete123"`	`web.cpp`	senha WPA2 (mínimo de 8 caracteres, senão o AP não sobe)
`HTTP_PORTA`	80	`web.cpp`	página do app: `http://192.168.4.1`
`WS_PORTA`	81	`web.cpp`	WebSocket: `ws://192.168.4.1:81/`

Valores de fábrica dos parâmetros ajustáveis

Usados no primeiro boot, no comando `reset` e no botão "Restaurar padrão" do app. São folgados de propósito: funcionam sem calibração fina. As faixas aceitas são impostas por `validarParams` (posture).

Constante	Valor	Unidade	Campo em `Params`	Faixa aceita
`PADRAO_LIMITE_GRAUS`	20	°	`limiteGraus`	2–60
`PADRAO_HISTERESE`	3	° (MPU) / pontos % (flex)	`histerese`	0 até (menor limite − 1)
`PADRAO_TEMPO_MS`	3000	ms	`tempoMs`	0–60000
`PADRAO_LIMITE_FLEX`	15	%	`limiteFlex`	2–90
`PADRAO_INTENSIDADE`	100	% do duty	`intensidade`	0–100

Constante	Valor	Usada em	Descrição
`FIRMWARE_VERSAO`	`"1.0.0"`	`main.cpp`, `protocolo.cpp`	mostrada no boot da serial e enviada no `{"t":"config"}`

Armadilhas e cuidados

- **Números de linha citados por fora.** O `VALIDACAO.md` (roteiro do teste no hardware) cita linhas deste arquivo (`src/config.h:25`, `:27`, `:31`, `:39`, `:59`, `:65`). Não inserir nem remover linhas antes da seção do Wi-Fi; comentário novo vai no fim de uma linha existente ou depois da seção do Wi-Fi. Duas dessas citações já estavam deslocadas antes da documentação: `:31` aponta para `ADC_MIN_VALIDO` (o texto fala de `ADC_RUIDO_MAX`, linha 33) e `:39` aponta para um comentário (o texto fala de `MOTOR_LEDC_FREQ`, linha 44).
- **Pinos:** trocar um pino analógico para fora do ADC1 faz o sensor parar de funcionar com o Wi-Fi ligado.
- **`WIFI_SENHA` com menos de 8 caracteres:** `WiFi.softAP` falha e o colete fica sem app (continua vibrando).
- **`PERIODO_ADC_MS` × `ADC_MEDIA_N`:** juntos definem a janela da média (100 ms). Aumentar qualquer um deixa o flex mais lento para reagir.
- **`MPU_FILTRO_GIRO`:** mais perto de 1 = mais suave e mais lento para corrigir deriva; mais perto de 0 = mais rápido e mais sensível a vibração (inclusive a do próprio motor).
- **`FLEX_LIGA_MS` vale para os três sensores analógicos** (flex 1, flex 2 e FSR), apesar do nome.